Trabajo Práctico 02 — Procesos ETL

Bases de Datos Masivas (11088) — Departamento de Ciencias Básicas **2° Cuatrimestre 2026** — Universidad Nacional de Luján

Alumno/a: Federico Kasparian **Fecha de entrega:** 25/09/2026

1. Introducción

Este informe resuelve el TP02 sobre el dataset *Education Statistics* del World Bank Group (`API_4_DS2_en_csv_v2_3434744.csv`, 43.624 filas × 69 columnas: 4 columnas identificadoras y 64 columnas de años 1960–2023) aplicando los dos paradigmas de desarrollo ETL pedidos por la consigna:

- **Programático (code-first):** Python + pandas, en `TP02_ETL_Respuesta.ipynb`.
- **Visual/orquestado (GUI/low-code):** Apache Hop 2.19.0, dockerizado, en `hop/` (tres pipelines `.hpl` más un workflow `etl-tp02.hwf` que los encadena).

Ambos enfoques implementan exactamente la misma lógica de negocio y producen los mismos tres archivos de salida:

Archivo	Campos	Filas
`paises.csv`	`id, nombre_pais, codigo_pais`	266
`preguntas.csv`	`id, pregunta`	151
`educacion_primaria_2023.csv`	`pais, codigo_pais, duracion_primaria_anios`	255 (ordenado desc.)

Los tres pipelines de Hop se corrieron de punta a punta con `hop-run` (motor local, dockerizado) y los tres reconcilian **exactamente** con la salida de pandas: mismos 266/151/255 registros y mismos valores, verificado por comparación fila a fila entre ambas salidas. La única diferencia observada es el orden entre países empatados en la misma duración en `educacion_primaria_2023.csv` (ninguna de las dos implementaciones fija un criterio de desempate secundario, y la consigna no lo exige) — el conjunto de filas y el orden por duración descendente son idénticos.

2. Resumen de las transformaciones

Países. Se seleccionan `Country Name`/`Country Code`, se recortan espacios, se descartan nulos/vacíos en los campos base y se deduplica. El dataset del Banco Mundial no distingue países de entidades regionales agregadas (`Arab World`, `Euro area`, etc.) en la misma tabla; la consigna admite explícitamente esto último, así que las 266 filas resultantes incluyen ambas categorías.

Preguntas. Se toma `Indicator Name`, se elimina con una expresión regular (`\s\([^)]\)`, aplicada globalmente, no solo a la primera coincidencia) todo contenido entre paréntesis — incluidos los casos con **más de un** grupo de paréntesis en la misma cadena, como `"Unemployment, total (% of total labor force) (modeled ILO estimate)"` — y se hace trim. De los 164 indicadores originales, 13 colapsan en el mismo texto tras la limpieza (por ejemplo, `"School enrollment, primary, male (% gross)"` y `"(% net)"` del mismo indicador), quedando 151 preguntas únicas.

Educación primaria 2023. Se filtra el indicador `SE.PRM.DURS` (`"Primary education, duration (years)"`) y la columna `2023`. De los 266 países con ese indicador, 11 (4,1 %) no tienen dato reportado para 2023. **Se descartan** en lugar de imputar: la duración de la educación primaria es una decisión normativa de cada país —los años que fija su ley educativa—, no una medición continua con ruido de muestreo, así que no hay un valor plausible que inferir a partir de otros países o períodos. Al ser una

proporción baja (4,1 %), el descarte no compromete la representatividad del ranking. En la implementación de Hop esos 11 casos quedan visibles en `salida/03-sin-dato-2023.csv` en vez de perderse en silencio.

3. Preguntas conceptuales

3.1 Facilidad de desarrollo y curva de aprendizaje

Pandas resultó más rápido de escribir y, sobre todo, más rápido de *verificar*, para la limpieza con RegEx del Ejercicio 2. La expresión (`\s\([^)]\)` → `""`) se escribe una vez, y en la misma celda del notebook se arma una tabla `original → limpio` sobre seis casos reales que incluyen los indicadores con dos grupos de paréntesis —el caso límite de la consigna—, se corre y se lee el resultado en el momento. Iterar sobre la expresión regular (probarla, ver que rompe con paréntesis anidados o con espacios sobrantes, corregirla) es un ciclo de segundos porque el código, el dato y el resultado están en la misma pantalla.

En **Apache Hop** la misma transformación es un único *step* — *Replace in string*, con el casillero "usar RegEx" tildado — así que en términos de líneas de configuración es comparable o incluso más simple: no hay que escribir `re.sub` ni una función. Pero la curva de aprendizaje es distinta, no ausente: hay que saber *que* ese step existe (entre decenas de transforms con nombres parecidos — *Replace in string* vs *Regex evaluation* vs *String operations*), y verificar el resultado exige abrir el diálogo del step, tildar "Preview", ejecutar y leer una grilla aparte — no hay una vista donde el before/after conviva con la definición de la regla como en el notebook. Además, el `\s*` inicial (para no dejar un espacio colgado cuando se saca el paréntesis) es exactamente el tipo de detalle que en pandas se prueba al toque contra los seis ejemplos reales, y en Hop exige una corrida de Preview por cada ajuste.

Balance: para transformaciones de texto con RegEx, pandas gana en velocidad de iteración/depuración; Hop no es más difícil de *usar* una vez que se conoce el step correcto, pero la curva de aprendizaje inicial (encontrar el step, entender sus banderas) es más alta que leer la documentación de `re`.

3.2 Manejo de errores, trazabilidad y depuración

Acá la comparación favorece a **Apache Hop**, y la razón concreta apareció al construir el Ejercicio 3: la consigna pide "especificar la estrategia adoptada" para los 11 países sin dato 2023. En el notebook, "especificar la estrategia" significa **escribirlo en una celda Markdown** (la sección "Respuesta 3" de este informe) — el propio `.dropna()` no deja rastro físico de qué filas se descartaron, salvo que el desarrollador decida imprimir un conteo, como se hizo. En el pipeline `03-educacion-primaria-2023.hpl`, el segundo *Filter rows* tiene **dos salidas conectadas**: la verdadera sigue el flujo normal, y la falsa escribe `salida/03-sin-dato-2023.csv` con los 11 países excluidos. La trazabilidad no depende de que alguien se acuerde de loguearla: es parte de la estructura del pipeline, visible como una caja más en el diagrama.

La otra diferencia real es el **Preview por step**: clic derecho sobre cualquier transform → "Preview output" muestra las filas *tal como están en ese punto exacto* del flujo, sin necesitar variables intermedias. En pandas, inspeccionar un paso intermedio significa o bien no encadenar las operaciones (asignar a variables nombradas, como se hizo aquí: `paises`, luego filtrado, luego `paises` deduplicado) o interrumpir una cadena con `.pipe(lambda df: (print(df.head()), df)[1])` para no perder el dato. El notebook, a cambio, tiene algo que Hop no tiene de forma nativa: el **traceback de Python completo** ante un error (tipo, línea, valores involucrados) contra el log de Hop, que reporta el step y el mensaje pero no siempre el valor puntual que lo disparó.

Balance: Hop hace más *visible por diseño* el manejo de rechazos (dos salidas conectadas por step, sin código extra) y el estado intermedio (Preview sin instrumentar nada); pandas da más detalle en el

momento del error (traceback), pero la trazabilidad de las filas descartadas queda librada a que el desarrollador la instrumente a mano.

3.3 Escalabilidad y rendimiento en procesamiento masivo

pandas está limitado por diseño a un único proceso y a que el DataFrame completo entre en la memoria RAM de una sola máquina. Este dataset (43.624 × 69, ~22 MB) es trivial, pero la limitación no es de tamaño de archivo sino de **arquitectura**: `pd.read_csv` materializa todo el DataFrame antes de poder operar sobre él (salvo que se use `chunksize`, lo que obliga a reescribir cada transformación como un acumulador manual), no hay paralelismo entre máquinas, y duplicar el DataFrame en cada paso intermedio (algo que pandas hace con frecuencia por su API basada en copias) multiplica el uso de memoria. Para un dataset de cientos de GB, pandas deja de ser viable sin herramientas adicionales (Dask, Polars con *streaming*, o migrar a Spark).

Apache Hop no ejecuta las transformaciones en memoria como una tabla completa: cada transform procesa la fila apenas llega, en un modelo de streaming con buffers acotados (el *CSV file input* de este TP usa un `buffer_size` de 50.000 filas, no carga las 43.624 de una vez en una estructura tipo tabla). Eso ya lo hace más liviano para datasets grandes en una sola máquina. Pero la diferencia arquitectónica más importante para "Big Data" real es otra: **el mismo pipeline `.hpl` que se construye acá es agnóstico del motor de ejecución**. Hop puede correr un pipeline con su motor nativo (lo usado en este TP) o, sin tocar el diseño visual, delegar la ejecución a **Apache Spark, Apache Flink o Google Cloud Dataflow** a través de su integración con Apache Beam — el pipeline se traduce a un plan distribuido que particiona el trabajo entre nodos de un clúster. Es la diferencia entre escalar *verticalmente* (una máquina más grande, el límite de pandas) y escalar *horizontalmente* (más máquinas, lo que habilita Hop vía Beam) sin reescribir la lógica de negocio.

Balance: para el volumen de este TP, ambos procesan el dataset en segundos y la diferencia es irrelevante en la práctica. La distinción importa recién cuando el dataset no entra en RAM: ahí pandas exige reescribir el código (chunking manual o migrar de herramienta) mientras que Hop permite reapuntar el mismo diseño visual a un motor distribuido.

3.4 Portabilidad, automatización y despliegue en producción

La imagen `apache/hop-web:2.19.0` usada en este TP empaqueta el runtime completo — JVM, Tomcat, la GUI web y el motor de ejecución `hop-run` — en un único artefacto versionado. Eso tiene dos consecuencias prácticas concretas, visibles en `hop/docker-compose.yml` de este mismo TP: (1) levantar el entorno de desarrollo es `docker compose up`, sin instalar Java, Python ni ninguna librería en la máquina host; y (2) **el mismo contenedor que se usa para diseñar visualmente el pipeline es el que lo ejecuta en producción** vía `hop-run.sh --file=... --runconfig=...` (ver sección 4 del `README.md` de `hop/`), sin una capa de traducción entre "cómo lo armé" y "cómo corre". El pipeline y el workflow (`.hpl`/`.hwf`) son XML versionable en git, igual que el notebook, y el workflow (`etl-tp02.hwf`) ya resuelve la orquestación secuencial y el manejo de fallos (rama de éxito vs. rama de aborto) **sin depender de una herramienta externa**: es capacidad nativa del motor.

Un notebook o script de Python, para llegar al mismo nivel de madurez operativa, necesita **ensamblar piezas separadas**: un `pyproject.toml`/`requirements.txt` fijando versiones (y su propio riesgo de *drift* entre el entorno de desarrollo y el de producción), un orquestador externo (Airflow, Cron) que no viene con el código sino que hay que instalar, configurar y mantener aparte, y —si el pipeline debe correr sin intervención— convertir el notebook en script (`nbconvert`/`papermill`) porque un `.ipynb` no está pensado para ejecución desatendida. La ventaja de este enfoque no es despreciable: al ser Python estándar, se integra directamente con todo el ecosistema de datos/ML (bibliotecas de terceros, APIs, modelos) sin las limitaciones del catálogo de steps de una herramienta cerrada, y Airflow ofrece reintentos, alertas y un dashboard de ejecuciones que Hop no tiene *out of the box*.

Balance: Hop dockerizado ofrece menor costo de ensamblado para llegar a "pipeline reproducible y orquestado" porque el runtime y el orquestador (workflows) vienen integrados en la misma imagen; un ETL en Python/pandas llega al mismo punto con más piezas por conectar (gestor de entornos + orquestador externo + adaptación para ejecución desatendida), a cambio de mayor flexibilidad para integrarse con el resto del ecosistema Python.

4. Conclusión

Para este TP —un dataset que entra cómodo en memoria y tres transformaciones de complejidad moderada— ambos paradigmas llegan al mismo resultado con un esfuerzo comparable. Las diferencias reales no están en "cuál es mejor" sino en **qué problema resuelve nativamente cada uno**: pandas da el ciclo de iteración más corto para diseñar y depurar la lógica de negocio (todo en un mismo lugar, con trazas de error completas); Apache Hop da, sin código adicional, trazabilidad estructural de los rechazos, un camino directo hacia el procesamiento distribuido cuando el volumen deja de entrar en una sola máquina, y un runtime dockerizado que es a la vez el entorno de diseño y el de ejecución en producción.

Referencias

- World Bank Group. *Education Statistics — All Indicators*. <https://data.worldbank.org/topic/education?view=chart>
- Apache Hop Project. *Apache Hop Documentation*, incl. *Filter Rows*, *Replace in String*, *Add Sequence*, *CSV File Input*. <https://hop.apache.org/manual/latest/>
- Apache Hop Project. *Apache Beam engine* — ejecución distribuida de pipelines sobre Spark/Flink/Dataflow. <https://hop.apache.org/manual/latest/pipeline/beam/index.html>
- Kimball, R. y Caserta, J. (2004). *The Data Warehouse ETL Toolkit*. Wiley.